ELEC 475 - Lab 2 Post-Lab Report

Prepared By:

Mihailo Ratkov – 20395168

1. Architectural Specifications

Base LeNet-5:

```
==========================================================
Layer (type:depth-idx)           Output Shape      Param #
==========================================================
LeNet5                           [1, 10]           --
├─Sequential: 1-1                [1, 16, 13, 13]   --
│    └─Conv2d: 2-1               [1, 6, 60, 60]    456
│    └─Tanh: 2-2                 [1, 6, 60, 60]    --
│    └─AvgPool2d: 2-3            [1, 6, 30, 30]    --
│    └─Conv2d: 2-4               [1, 16, 26, 26]   2,416
│    └─Tanh: 2-5                 [1, 16, 26, 26]   --
│    └─AvgPool2d: 2-6            [1, 16, 13, 13]   --
├─Sequential: 1-2                [1, 10]           --
│    └─Linear: 2-7               [1, 120]          324,600
│    └─Tanh: 2-8                 [1, 120]          --
│    └─Linear: 2-9               [1, 84]           10,164
│    └─Tanh: 2-10                [1, 84]           --
│    └─Linear: 2-11              [1, 10]           850
==========================================================
Total params: 338,486
Trainable params: 338,486
Non-trainable params: 0
Total mult-adds (M): 3.61
==========================================================
Input size (MB): 0.05
Forward/backward pass size (MB): 0.26
```

```
Params size (MB): 1.35

Estimated total size (MB): 1.66

==========================================================
```

Modernized LeNet:

```
==========================================================================
Layer (type:depth-idx)                   Output Shape          Param #
==========================================================================
LeNetModern                              [1, 10]                   --
├─Sequential: 1-1                        [1, 16, 15, 15]           --
│    └─Conv2d: 2-1                       [1, 6, 64, 64]            456
│    └─ReLU: 2-2                         [1, 6, 64, 64]            --
│    └─MaxPool2d: 2-3                    [1, 6, 31, 31]            --
│    └─Conv2d: 2-4                       [1, 16, 31, 31]
2,416
│    └─ReLU: 2-5                         [1, 16, 31, 31]           --
│    └─MaxPool2d: 2-6                    [1, 16, 15, 15]           --
├─Sequential: 1-2                        [1, 10]                   --
│    └─Linear: 2-7                       [1, 120]
432,120
│    └─ReLU: 2-8                         [1, 120]                  --
│    └─Linear: 2-9                       [1, 84]
10,164
│    └─ReLU: 2-10                        [1, 84]                   --
│    └─Linear: 2-11                      [1, 10]                   850
==========================================================================
Total params: 446,006

Trainable params: 446,006

Non-trainable params: 0
```

```
Total mult-adds (Units.MEGABYTES): 4.63
==========================================================================
Input size (MB): 0.05
Forward/backward pass size (MB): 0.32
Params size (MB): 1.78
Estimated Total Size (MB): 2.15
==========================================================================
```

Full AlexNet-64:

```
==========================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================
AlexNet64                                [1, 10]                   --
├─Sequential: 1-1                        [1, 256, 3, 3]            --
│    └─Conv2d: 2-1                       [1, 64, 32, 32]         4,864
│    └─BatchNorm2d: 2-2                  [1, 64, 32, 32]           128
│    └─ReLU: 2-3                         [1, 64, 32, 32]           --
│    └─MaxPool2d: 2-4                    [1, 64, 15, 15]           --
│    └─Conv2d: 2-5                       [1, 192, 15, 15]      307,392
│    └─BatchNorm2d: 2-6                  [1, 192, 15, 15]          384
│    └─ReLU: 2-7                         [1, 192, 15, 15]          --
│    └─MaxPool2d: 2-8                    [1, 192, 7, 7]            --
│    └─Conv2d: 2-9                       [1, 384, 7, 7]        663,936
│    └─BatchNorm2d: 2-10                 [1, 384, 7, 7]            768
│    └─ReLU: 2-11                        [1, 384, 7, 7]            --
│    └─Conv2d: 2-12                      [1, 384, 7, 7]      1,327,488
│    └─BatchNorm2d: 2-13                 [1, 384, 7, 7]            768
│    └─ReLU: 2-14                        [1, 384, 7, 7]            --
```

```
│    └─Conv2d: 2-15                       [1, 256, 7, 7]        884,992
│    └─BatchNorm2d: 2-16                  [1, 256, 7, 7]            512
│    └─ReLU: 2-17                         [1, 256, 7, 7]            --
│    └─MaxPool2d: 2-18                    [1, 256, 3, 3]            --
├─Sequential: 1-2                        [1, 10]                   --
│    └─Dropout: 2-19                      [1, 2304]                 --
│    └─Linear: 2-20                       [1, 1024]           2,360,320
│    └─ReLU: 2-21                         [1, 1024]                 --
│    └─Dropout: 2-22                      [1, 1024]                 --
│    └─Linear: 2-23                       [1, 1024]           1,049,600
│    └─ReLU: 2-24                         [1, 1024]                 --
│    └─Linear: 2-25                       [1, 10]                10,250
==========================================================================
Total params: 6,611,402
Trainable params: 6,611,402
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 218.51
==========================================================================
Input size (MB): 0.05
Forward/backward pass size (MB): 2.56
Params size (MB): 26.45
Estimated Total Size (MB): 29.05
==========================================================================
```

PyTorch Model Zoo AlexNet Baseline:

```
==========================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================
```

```
AlexNet                                  [1, 10]                   --
├─Sequential: 1-1                        [1, 256, 1, 1]            --
│    └─Conv2d: 2-1                       [1, 64, 15, 15]
23,296
│    └─ReLU: 2-2                         [1, 64, 15, 15]           --
│    └─MaxPool2d: 2-3                    [1, 64, 7, 7]             --
│    └─Conv2d: 2-4                       [1, 192, 7, 7]
307,392
│    └─ReLU: 2-5                         [1, 192, 7, 7]            --
│    └─MaxPool2d: 2-6                    [1, 192, 3, 3]            --
│    └─Conv2d: 2-7                       [1, 384, 3, 3]
663,936
│    └─ReLU: 2-8                         [1, 384, 3, 3]            --
│    └─Conv2d: 2-9                       [1, 256, 3, 3]
884,992
│    └─ReLU: 2-10                        [1, 256, 3, 3]            --
│    └─Conv2d: 2-11                      [1, 256, 3, 3]
590,080
│    └─ReLU: 2-12                        [1, 256, 3, 3]            --
│    └─MaxPool2d: 2-13                   [1, 256, 1, 1]            --
├─AdaptiveAvgPool2d: 1-2                 [1, 256, 6, 6]            --
├─Sequential: 1-3                        [1, 10]                   --
│    └─Dropout: 2-14                     [1, 9216]                 --
│    └─Linear: 2-15                      [1, 4096]
37,752,832
│    └─ReLU: 2-16                        [1, 4096]                 --
│    └─Dropout: 2-17                     [1, 4096]                 --
│    └─Linear: 2-18                      [1, 4096]
16,781,312
```

```
│    └─ReLU: 2-19                        [1, 4096]                 --
│    └─Linear: 2-20                      [1, 10]
40,970
==========================================================================
Total params: 57,044,810
Trainable params: 57,044,810
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 94.13
==========================================================================
Input size (MB): 0.05
Forward/backward pass size (MB): 0.32
Params size (MB): 228.18
Estimated Total Size (MB): 228.55
==========================================================================
```

2 Empirical Results and Figures

2.1 Master Ablation Summary Tables

Table 1: Architectural Configuration & Computational Complexity Profile (mean epoch time when training on CIFAR-10)

Model Architecture	**Conv Depth**	**Normalization & Regularization**	**Data Augmentation**	**Trainable Params (M)**	**FLOPs (MFLOPs)**	**Mean Epoch Time (s)**	**Inference Latency (ms)**
1. Base LeNet-5	2	None	None	0.338	7.17	75.411	0.71
2. Modernized LeNet	2	None (ReLU + Overlapping MaxPool)	None	0.446	9.19	66.404	1.03
3. AlexNet (No Norm)	5	Dropout (p =0.5)	None	6.609	436.70	34.673	1.84

4. AlexNet + LRN	5	LRN + Dropout (p = 0.5)	None	6.609	436.70	50.935	2.13
5. AlexNet + BatchNorm	5	BatchNorm2d + Dropout (p=0.5)	None	6.611	437.97	38.189	2.54
6. AlexNet + BN + Aug	5	BatchNorm + Dropout (p = 0.5)	Pure Pytorch Aug	6.611	437.97	66.750	2.54
7. Pytorch Zoo Alexnet	5	PyTorch Zoo (AdaptiveAvgPool 2d)	None	57.045	188.20	89.898	2.96

Table 2: Empirical Predictive Performance Across Datasets (separate training runs from 3 Technical Analysis and Discussion)

Model Architecture	CIFAR-10 Val Acc (%)	CIFAR-10 Val Macro F1 (%)	Tiny ImageNet Val Acc (%)	Tiny ImageNet Val Macro F1 (%)
1. Base LeNet-5	56.46%	56.10%	14.77%	12.29%
2. Modernized LeNet	69.64%	69.79%	27.83%	26.05%
3. AlexNet (No Norm)	74.62%	74.52%	17.90%	15.03%
4. AlexNet + LRN	74.06%	73.96%	18.70%	15.91%
5. AlexNet + BatchNorm	78.64%	78.84%	36.36%	34.98%
6. AlexNet + BN + Aug	80.44%	80.20%	34.24%	32.37%
7. Pytorch Zoo Alexnet	66.03%	65.90%	12.74%	10.06%

2.2 Training and Validation Loss/Accuracy Curves

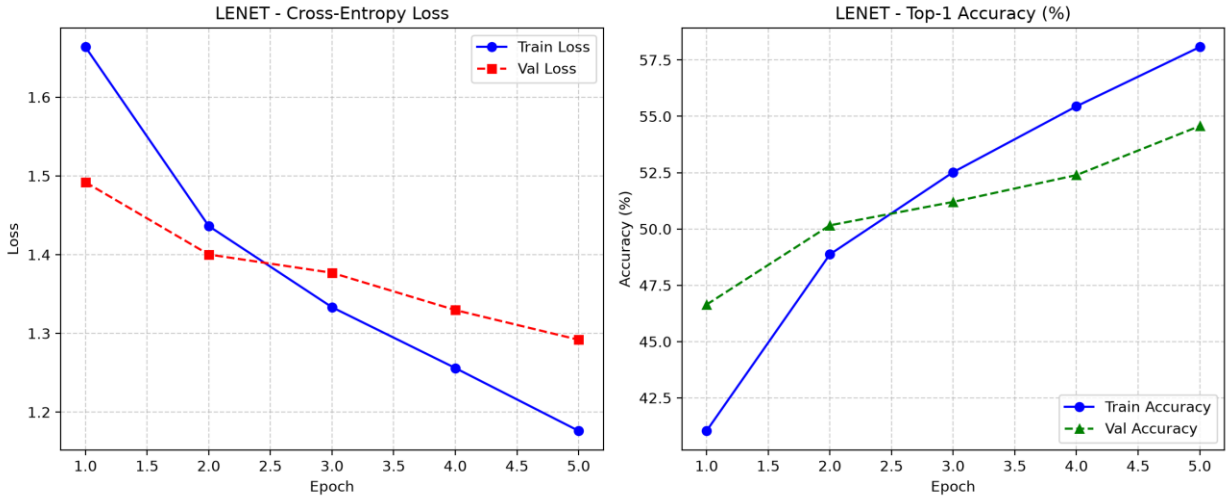


Figure 1: Training and validation loss/accuracy curve of LeNet

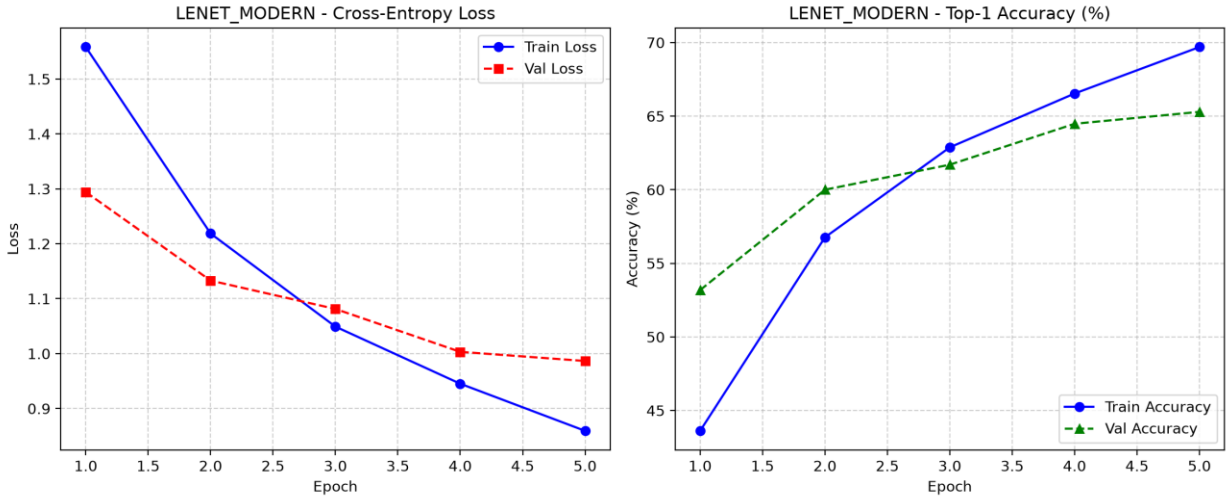


Figure 2: Training and validation loss/accuracy curve of Modernized LeNet

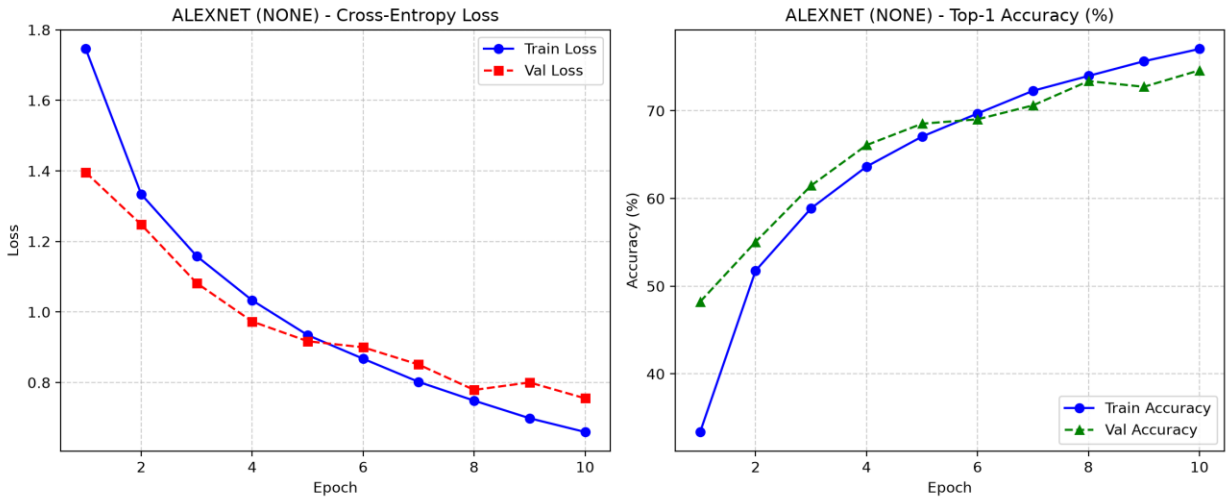


Figure 3: Training and validation loss/accuracy curve of AlexNet without normalization

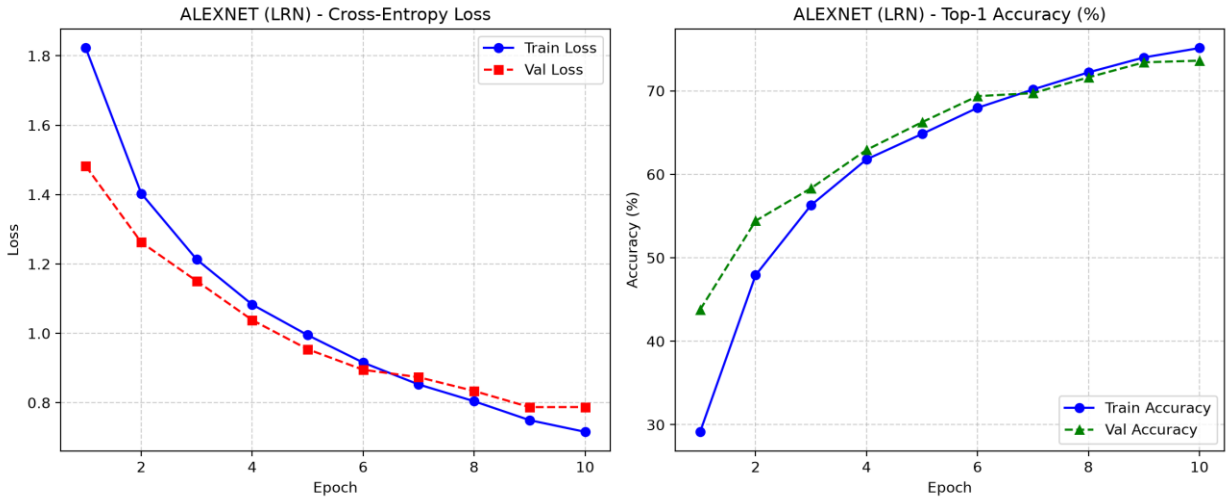


Figure 4: Training and validation loss/accuracy curve of AlexNet with local response normalization

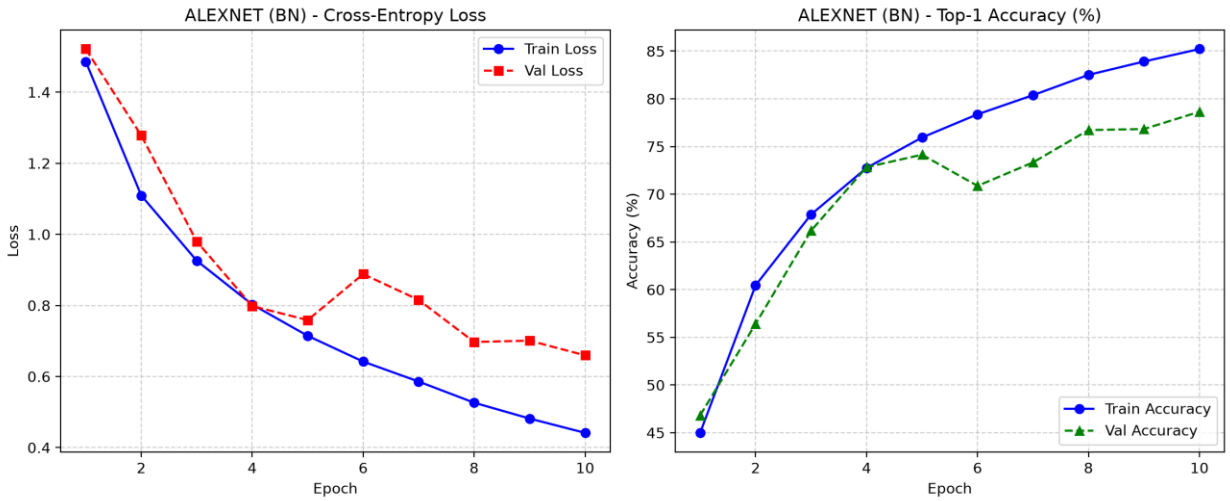


Figure 5: Training and validation loss/accuracy curve of AlexNet with batch normalization

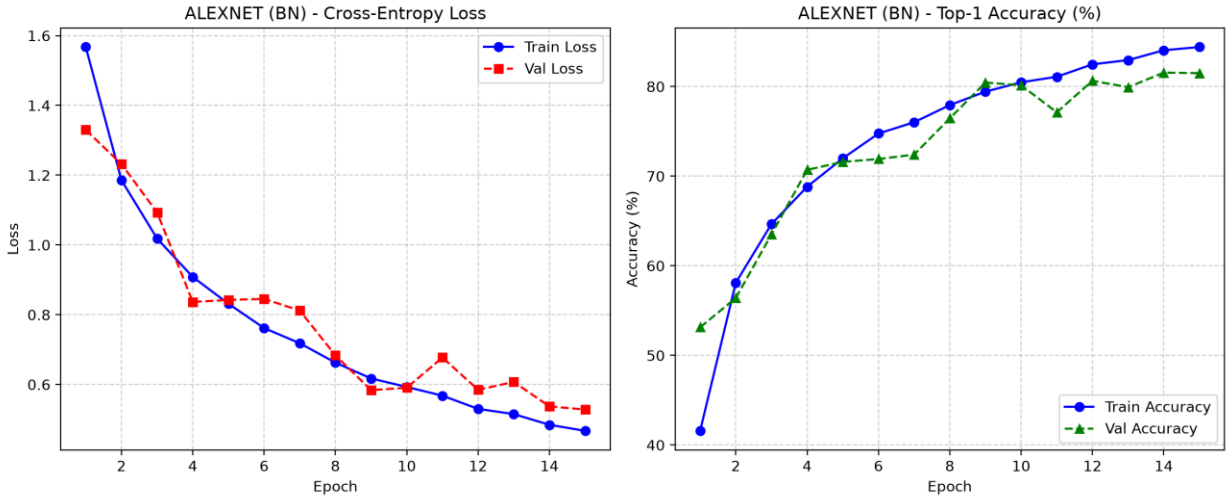


Figure 6: Training and validation loss/accuracy curve of AlexNet with batch normalization with data augmentation

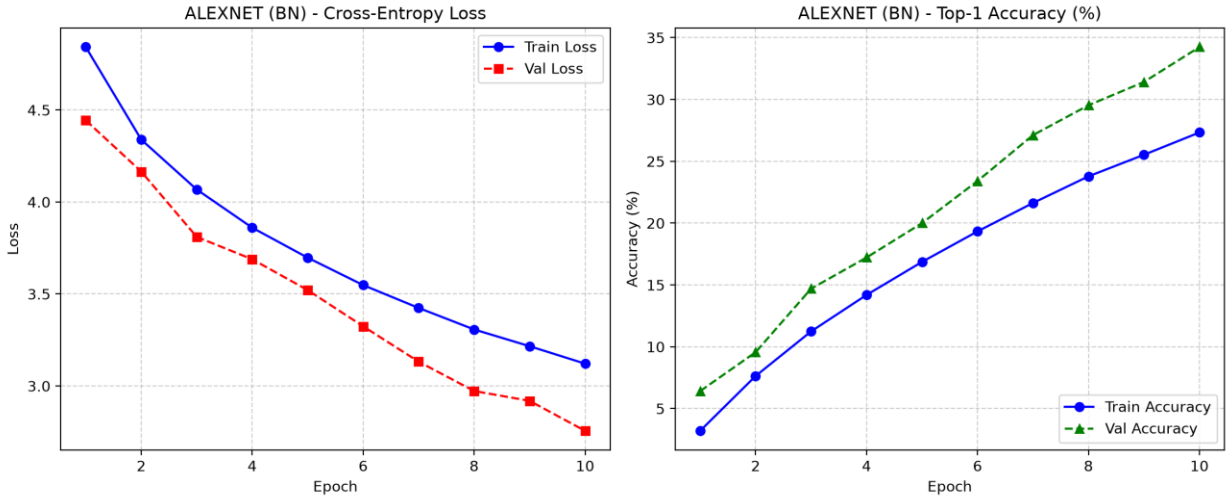


Figure 7: Training and validation loss/accuracy curve of AlexNet with batch normalization with data augmentation trained on Tiny ImageNet

2.3 Output from demo_transforms.py

```
======================================================================

ELEC 475: Transform Paradigm & Variable Type Diagnostic

======================================================================


[1] OpenCV (cv2) Output:

    - Object Type      : <class 'numpy.ndarray'>
```

```
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Width] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cuda:0 (NVIDIA GPU VRAM)

----------------------------------------------------------------------
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
----------------------------------------------------------------------
- OpenCV (CPU):   21.66 ms  ( 46,167 img/s) + Host-to-Device Copy required
```

```
- NumPy (CPU):   52.40 ms  ( 19,084 img/s) + Host-to-Device Copy required

- PyTorch (GPU Tensors):  668.86 ms  (  1,495 img/s) [ZERO PCIe Transfer Overhead]

================================================================================
```

2.3 Class Balance & Feature Distribution Figures

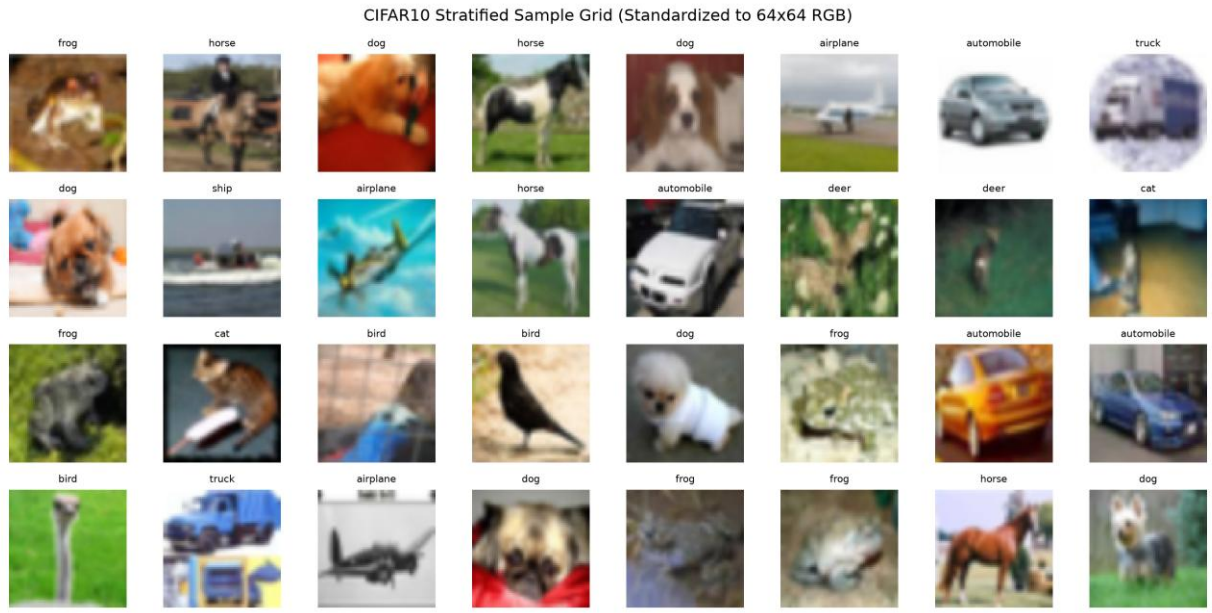


Figure 8: CIFAR-10 sample distribution

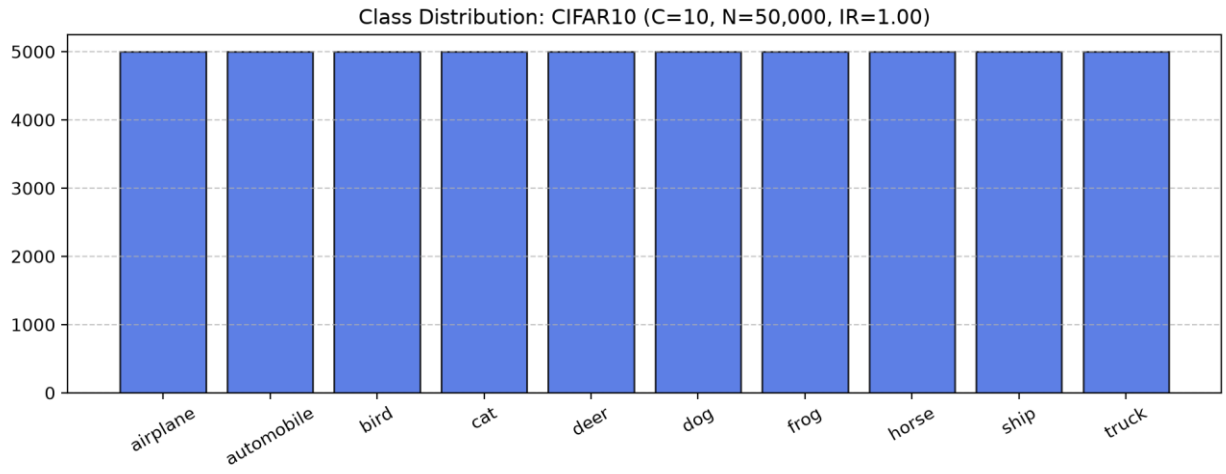


Figure 9: Class distribution of CIFAR-10

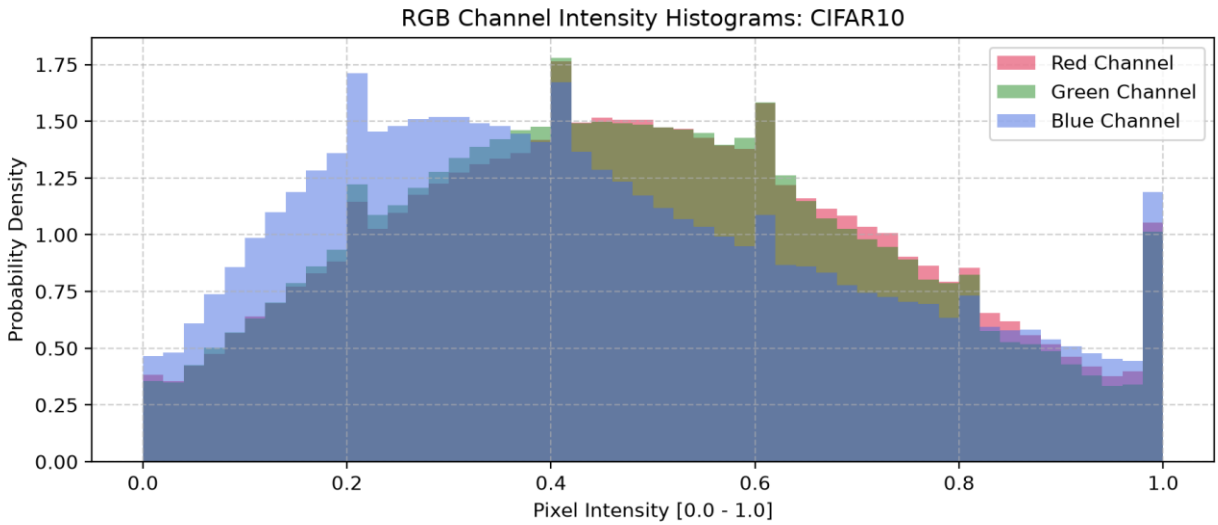


Figure 10: Channel histogram of CIFAR-10



Figure 11: Tiny ImageNet sample distribution

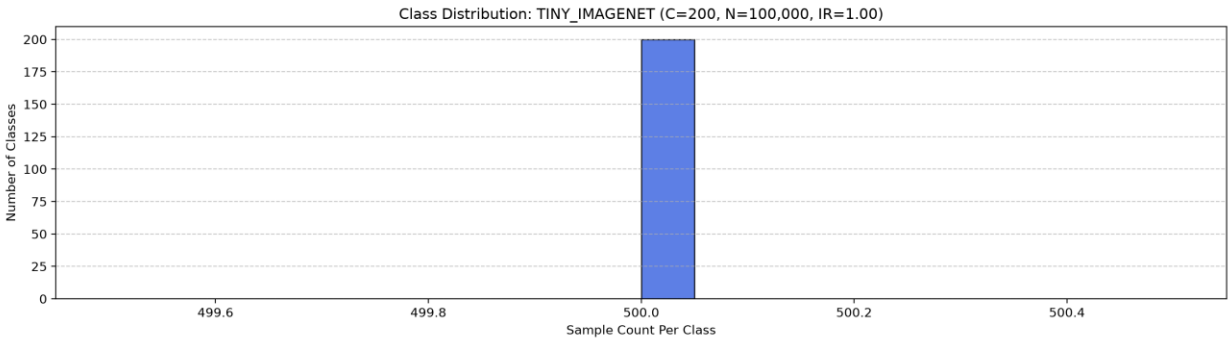


Figure 12: Class distribution of Tiny ImageNet

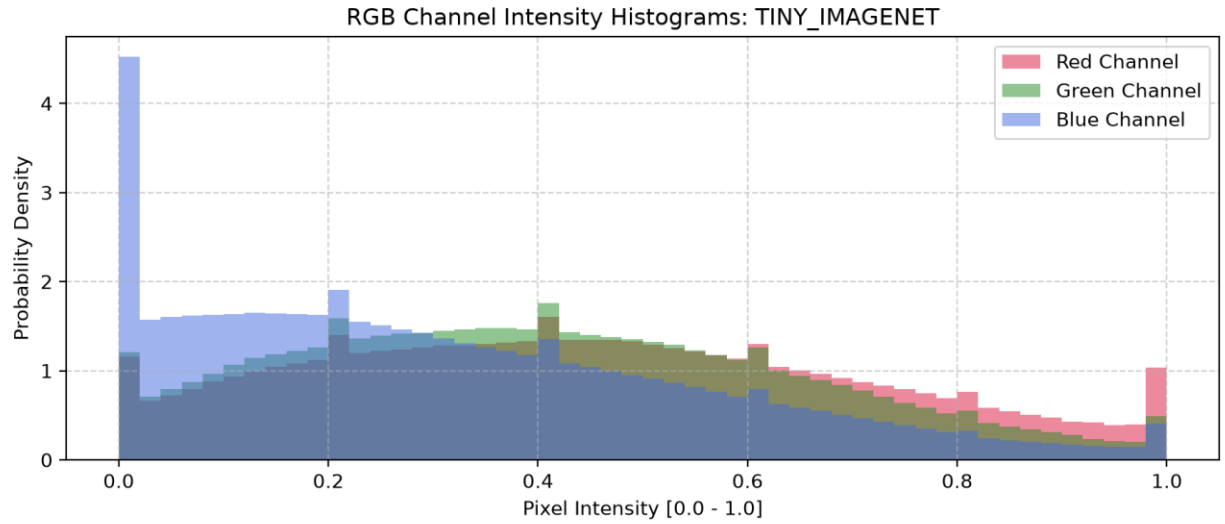


Figure 13: Channel histogram of CIFAR-10

2.4 Terminal Output Tables

```
======================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device         : cuda (NVIDIA GeForce RTX 4050 Laptop GPU)
Dataset        : CIFAR10 (10 classes)
Input Shape    : (1, 3, 64, 64)
======================================================================

Instantiating models for profiling...
Profiling FLOPs, Multiply-Accumulates, and Latency...
```

Model Architecture	Params (M)	MACs (M)	FLOPs (M)	Latency (ms)	Throughput (img/s)
1. LeNet-5 (Base)	0.338	3.59	7.17	0.62	1617.1
2. LeNet-Modern (ReLU+Pool)	0.446	4.59	9.19	0.89	1124.4
3. AlexNet-64 (None)	6.609	218.35	436.70	1.21	827.1
4. AlexNet-64 (LRN)	6.609	218.35	436.70	1.22	821.5
5. AlexNet-64 (BatchNorm)	6.611	218.98	437.97	1.21	823.8
6. Torchvision Zoo AlexNet	57.045	94.10	188.20	2.66	375.3

==========

Academic Notes on Complexity Metrics:

1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point Operations (FLOPs).

2. Memory vs Compute: Dense Linear layers contribute the majority of parameters, but Conv layers dominate FLOPs.

3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).

==========

==========

ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark

Device : cuda (NVIDIA GeForce RTX 4050 Laptop GPU)

Dataset : TINY_IMAGENET (200 classes)

Input Shape   : (1, 3, 64, 64)

==========

Instantiating models for profiling...

Profiling FLOPs, Multiply-Accumulates, and Latency...

Model Architecture	Params (M)	MACs (M)	FLOPs (M)	Latency (ms)	Throughput (img/s)
1. LeNet-5 (Base)	0.355	3.60	7.20	0.55	1817.9
2. LeNet-Modern (ReLU+Pool)	0.462	4.61	9.22	0.44	2274.3
3. AlexNet-64 (None)	6.804	218.54	437.08	0.73	1360.5
4. AlexNet-64 (LRN)	6.804	218.54	437.08	1.41	706.9
5. AlexNet-64 (BatchNorm)	6.806	219.18	438.36	1.36	736.2
6. Torchvision Zoo AlexNet	57.823	94.88	189.76	2.47	405.2

==========

Academic Notes on Complexity Metrics:

1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point Operations (FLOPs).

2. Memory vs Compute: Dense Linear layers contribute the majority of parameters, but Conv layers dominate FLOPs.

3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).

==========

3 Technical Analysis and Discussion

3.1 In-Lab Questions

Step 2.3:

Exact Parameter Count: 338,486

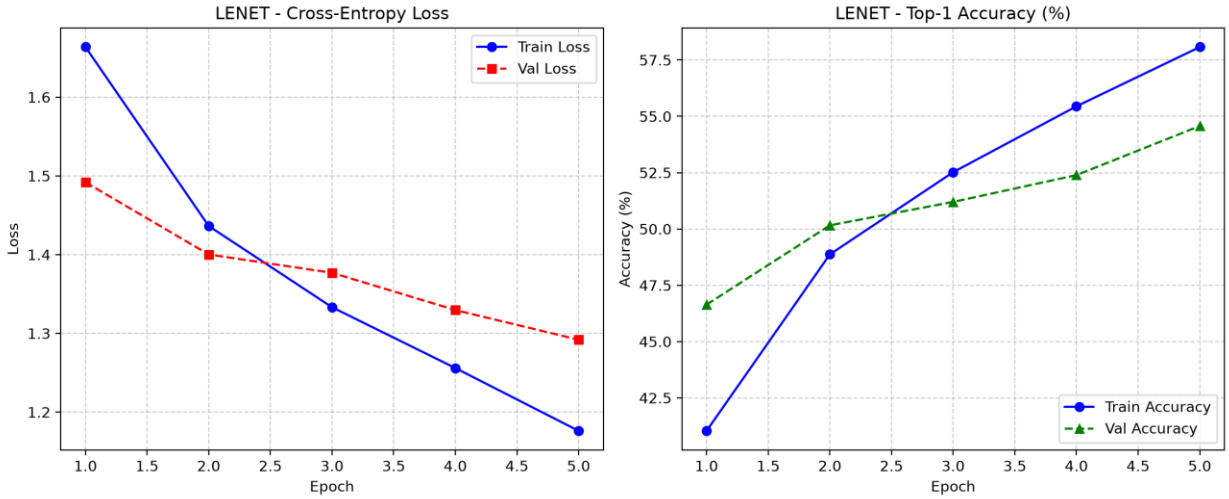
MACs: Multiply-Accumulates: 3,585,904 (3.59 MMACs)

FLOPs: Floating Point Ops: 7,171,808 (7.17 MFLOPs)

Step 3:

Empirical Validation Accuracy: Best Val Accuracy: 54.57%

Best Val Macro F1: 54.25%



Step 4:

Total params: 446,006

Multiply-Accumulates : 4,592,520 (4.59 MMACs)

Floating Point Ops : 9,185,040 (9.19 MFLOPs)

Step 5:

Final Validation Loss: 0.9865

Best Val Accuracy: 65.29%

Best Val Macro F1: 65.17%

FLOPs after 5 epochs = 9.19M FLOPs

Step 5 Questions:

Did your Modernized LeNet achieve higher validation accuracy and Macro than the baseline within 5 epochs? (**Yes** / No).

Q: Explain the underlying architectural and mathematical reasons. How did substituting saturating Tanh with non-saturating ReLU activations eliminate vanishing gradients, and how did overlapping max pooling preserve salient localized edge features?

A: ReLU activations cause the gradients to propagate fully to the earlier layers while the Tanh gradients vanish earlier, allowing ReLU to converge earlier. Max pooling combined with padding allows for edges to be passed through the pooling stages.

◦ Did the modernized architecture require higher computational resources (FLOPs / epoch execution time)? (**Yes** / No).

Q: ◦ Report your observed speed and throughput: Did the increase in arithmetic FLOPs noticeably slow down epoch wall-clock seconds or batch throughput on your hardware?

Speed (LeNet): 229.80s

Average Throughput (LeNet): 1157.5 s/s

Speed (Modern LeNet): 176.07s

Average Throughput (Modern LeNet): 1458.98 s/s

Q: Is the heavier model trade-off worthwhile? Where is Modernized LeNet clearly better (e.g., substantially faster convergence and higher classification accuracy with negligible latency penalty on modern GPUs), and where might the increased FLOPs and parameter footprint be a disadvantage (e.g., ultra-low-power edge microcontrollers with strict SRAM limits)?

A: The heavier model converges much faster than the lighter LeNet variant. Modernized LeNet attains higher F1 and training/testing accuracy in less epochs. Increased FLOPs may be a disadvantage on slower and lower power processors like edge microcontrollers.

Q: Modern Hardware Context: Why do small arithmetic FLOP differences matter far less on parallel modern GPUs with Tensor Cores (which thrive on saturating compute pipelines) than on sequential CPU or edge microcontroller cores?

A: Computing the weights on GPUs is largely faster because it can conduct matrix operations in parallel. This computation method vastly increases the speed and is majorly responsible for the performance between it and the CPU. As such, increasing the FLOPs would not contribute as much to the speed of the calculations.

Q: Observe the validation loss trajectories over the 5 epochs. Has the validation loss plateaued or is it continuing to descend? Based on this trajectory, would an Early Stopping controller halt training or allow it to proceed further?

A:

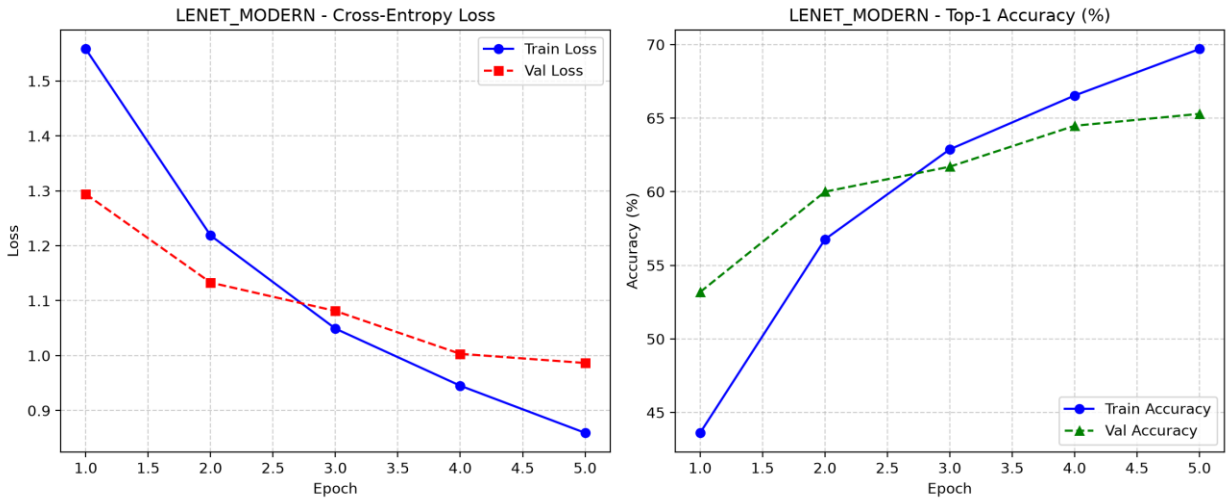


Figure 14: Modern LeNet loss curves

The loss has not fully plateaued yet so an early stopping controller would not halt training. It would likely proceed for around 3 more epochs.

3.2 Post-Lab Questions

3.2.1 Step 1:

==========

Layer (type:depth-idx) Output Shape Param #

==========

AlexNet64 [1, 10] --

├─Sequential: 1-1 [1, 256, 3, 3] --

│ └─Conv2d: 2-1 [1, 64, 32, 32] 4,864

│ └─BatchNorm2d: 2-2 [1, 64, 32, 32] 128

```
│    └─ReLU: 2-3                   [1, 64, 32, 32]       --
│    └─MaxPool2d: 2-4              [1, 64, 15, 15]       --
│    └─Conv2d: 2-5                 [1, 192, 15, 15]      307,392
│    └─BatchNorm2d: 2-6            [1, 192, 15, 15]      384
│    └─ReLU: 2-7                   [1, 192, 15, 15]      --
│    └─MaxPool2d: 2-8              [1, 192, 7, 7]        --
│    └─Conv2d: 2-9                 [1, 384, 7, 7]        663,936
│    └─BatchNorm2d: 2-10           [1, 384, 7, 7]        768
│    └─ReLU: 2-11                  [1, 384, 7, 7]        --
│    └─Conv2d: 2-12                [1, 384, 7, 7]        1,327,488
│    └─BatchNorm2d: 2-13           [1, 384, 7, 7]        768
│    └─ReLU: 2-14                  [1, 384, 7, 7]        --
│    └─Conv2d: 2-15                [1, 256, 7, 7]        884,992
│    └─BatchNorm2d: 2-16           [1, 256, 7, 7]        512
│    └─ReLU: 2-17                  [1, 256, 7, 7]        --
│    └─MaxPool2d: 2-18             [1, 256, 3, 3]        --
├─Sequential: 1-2                  [1, 10]               --
│    └─Dropout: 2-19               [1, 2304]             --
│    └─Linear: 2-20                [1, 1024]             2,360,320
│    └─ReLU: 2-21                  [1, 1024]             --
│    └─Dropout: 2-22               [1, 1024]             --
│    └─Linear: 2-23                [1, 1024]             1,049,600
│    └─ReLU: 2-24                  [1, 1024]             --
│    └─Linear: 2-25                [1, 10]               10,250
==========================================================================

Total params: 6,611,402
```

Trainable params: 6,611,402

Non-trainable params: 0

Total mult-adds (Units.MEGABYTES): 218.51

==========

Input size (MB): 0.05

Forward/backward pass size (MB): 2.56

Params size (MB): 26.45

Estimated Total Size (MB): 29.05

3.2.2 Step 2:

Q1: Did incorporating Batch Normalization yield an improvement in validation accuracy and Macro F1 over No Normalization and LRN?

A1: Yes

None = Best Val Accuracy: 74.62% | Best Val Macro F1: 74.52%

LRN = Best Val Accuracy: 73.61% | Best Val Macro F1: 73.50%

BN = Best Val Accuracy: 78.64% | Best Val Macro F1: 78.84%

Q2: Explain the underlying mathematical and optimization reasons. How does normalizing mini-batch activations reduce internal covariate shift and smooth the optimization landscape?

A2: Batch normalization (BN) outperforms no normalization and local response normalization by smoothing out the optimization landscape. This means the models without BN are more sensitive to the initialization of the model parameters and the learning rate, tending to converge slower and fall into suboptimal minima.

Mathematically, BN calculates the batch mean and variance and normalizes activations with the equation below:

$$\widehat{x_i} = \frac{x_i - \mu_B}{\sqrt{\sigma^2{}_B + \epsilon}} \quad (1)$$

Subsequently, a scale and shift is applied to the normalized activation:

$$y = \gamma\hat{x} + \beta \quad (2)$$

Applying these two equations means each layer gets 0-mean, unit variance inputs, with the scale and shift storing the original scale of the input.

BN normalizing the mean and variance keeps layers more stable and independent from each other. This solves the problem other models face where each layer's input distribution changes as earlier layers update, causing the later layers to chase the discrepancy in the early layers. Normalizing also smooths the optimization landscape by limiting how fast the gradients change. This allows for a larger learning rate without diverging from a minimum.

Q3: Did the addition of LRN or BatchNorm2d measurably alter your training epoch wall-clock time and arithmetic FLOPs?

A3: No

None = Training completed in 265.30s, Model Complexity: 6.609M Parameters | 436.70M FLOPs

LRN = Training completed in 323.64s, Model Complexity: 6.609M Parameters | 436.70M FLOPs

BN = Training completed in 266.21s, Model Complexity : 6.611M Parameters | 437.97M FLOPs

In terms of the overhead element-wise normalization adds to the model, the model does not converge faster than without normalization, but it is able to perform more FLOPs throughout training in the same amount of time. The BN model also converges to a higher training accuracy meaning that in the same amount of time as the model without normalization, it can determine a more optimal minimum. Overall, if sufficient memory is available for the overhead, then it should be considered for more optimal training.

Q4: Compare the variance and smoothness of the training loss trajectories across the three variants. Which normalization scheme demonstrated the most stable gradient progression?

A4:

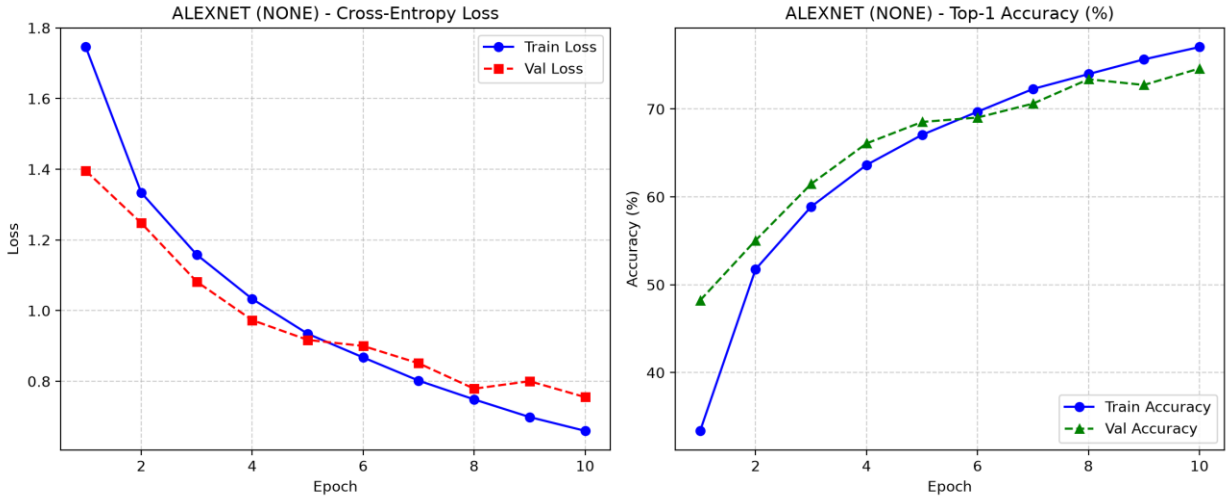


Figure 15: Training and validation loss curves for Alexnet without normalization

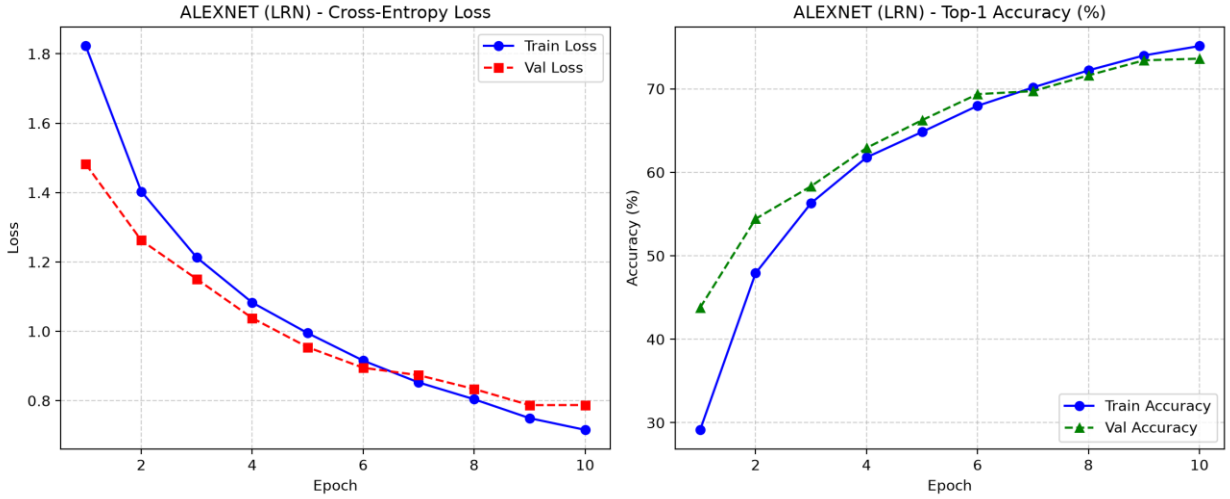


Figure 16: Training and validation loss curves for Alexnet with local response normalization

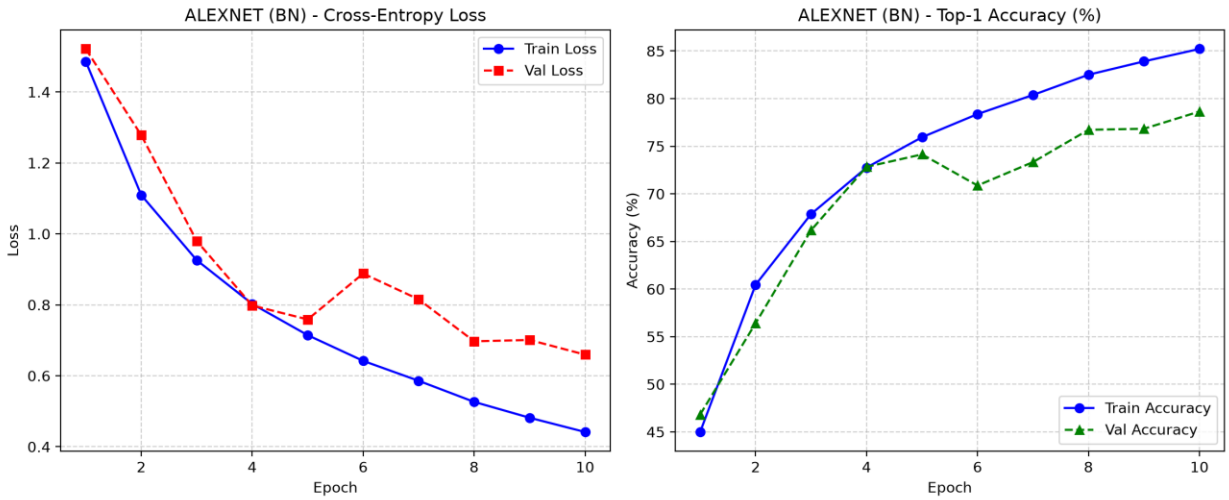


Figure 17: Training and validation curves for Alexnet with batch normalization

For training loss, all three variants smoothly converge at the same rate to a minimum over 10 training epochs. In terms of gradient progression, both LRN and BN exhibited nearly identical training loss trajectories. However, when comparing validation loss curves, batch normalization seems to be experiencing overfitting while LRN closely follows the training curve.

3.2.3 Step 3:

Q1: Did applying the PyTorch data augmentation pipeline reduce the generalization gap delta (Yes/No)? Report your measured training accuracy, validation accuracy, and calculated delta for both the unaugmented run and the augmented run.

A1: Yes

Delta = Training Accuracy – Validation Accuracy

Unaugmented Training Accuracy = 90.67

Unaugmented Validation Accuracy = 80.50

Unaugmented Delta = 90.67 - 80.50 = 10.17

Augmented Training Accruacy = 84.40

Augmented Validation Accuracy = 81.49

Augmented Delta = 84.40 - 81.49 = 2.91

Q2: Did data augmentation lead to an overall increase in final validation accuracy and Macro F1?

A2: Marginal

Unaugmented Validation Accuracy = 80.94%

Unaugmented Macro F1 = 80.84%

Augmented Validation Accuracy = 81.55%

Augmented Macro F1 = 81.53%

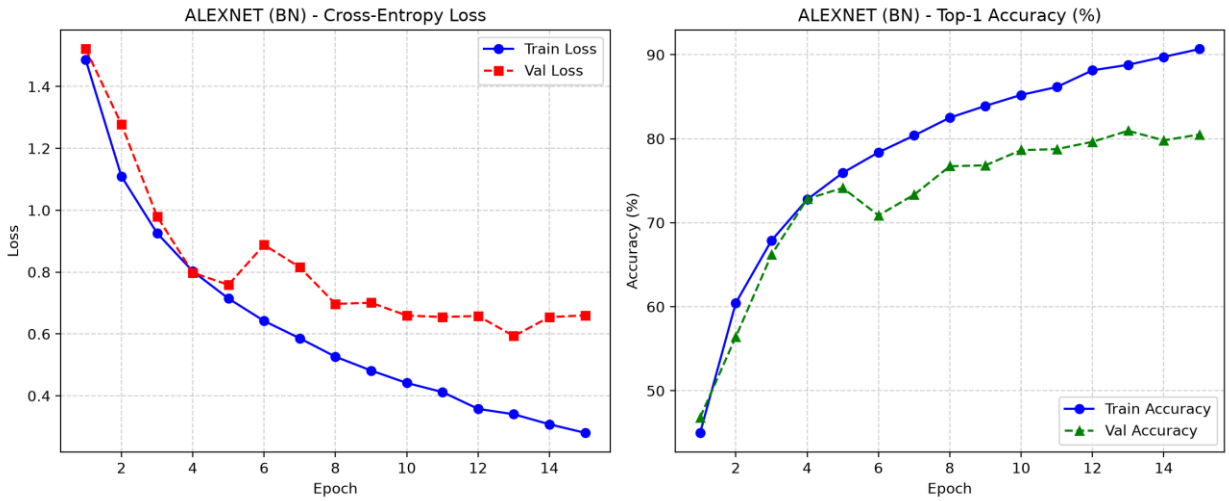


Figure 18: Train and validation curves for Alexnet with BN and no data augmentation

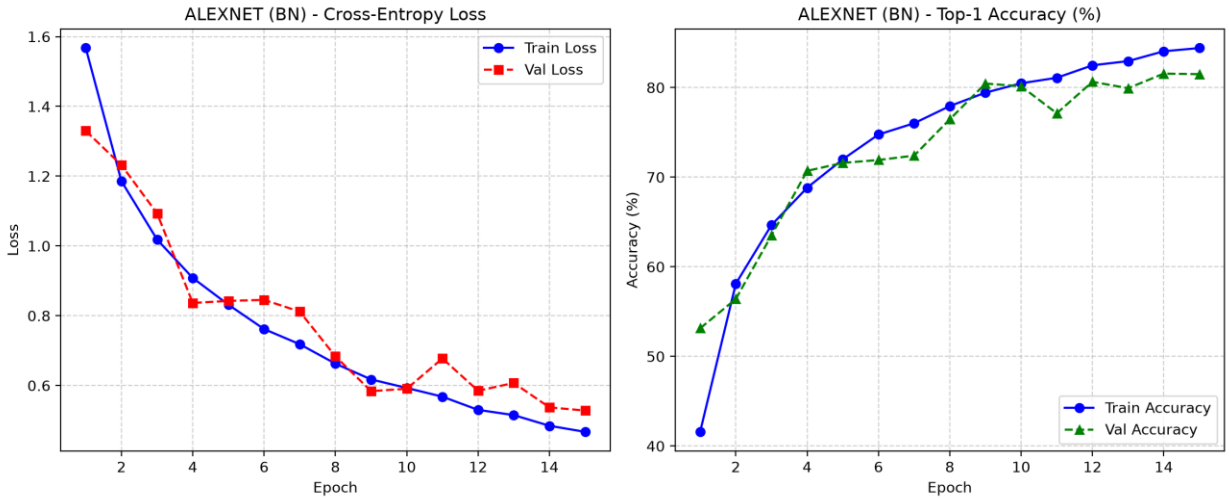


Figure 19: Train and validation curves for Alexnet with BN and data augmentation

From the plots, it becomes clear that the model training on augmented data simply required more epochs to fully assimilate the distribution. With more epochs, the validation accuracy and macro F1 would likely see greater improvements. This is due to the training data transformations in transforms.py hugely expanding the size of the training set.

Consequently, many more training epochs are necessary to allow the model to fully generalize and more accurately classify validation data.

3.2.4 Step 4:

Q1: Did AlexNet learn discriminative features on Tiny ImageNet significantly above the random guess baseline (1/200 or 0.5%)?

A1: Yes

Tiny ImageNet has 200 classes meaning the random guess baseline is 1/200. The trained model makes predications at far better accuracy than a random guess, evidenced by the 34.24% validation accuracy in Figure 20.

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)	Val F1 (%)	Time (s)	Throughput
1	4.8421	3.20	4.4439	6.41	3.76	129.34	773.1 s/s
2	4.3389	7.63	4.1652	9.56	7.47	160.87	621.6 s/s
3	4.0677	11.24	3.8102	14.70	11.59	108.36	922.9 s/s
4	3.8602	14.19	3.6886	17.22	15.13	84.14	1188.4 s/s
5	3.6971	16.87	3.5209	20.00	17.62	82.93	1205.9 s/s
6	3.5481	19.33	3.3239	23.40	21.82	150.49	664.5 s/s
7	3.4240	21.63	3.1320	27.12	24.96	171.78	582.1 s/s
8	3.3070	23.78	2.9725	29.53	26.99	172.23	580.6 s/s
9	3.2161	25.53	2.9191	31.42	29.61	126.03	793.5 s/s
10	3.1211	27.33	2.7559	34.24	32.37	127.90	781.9 s/s

Figure 20: Training and validation accuracies across 10 epochs

Q2: How do your validation accuracy and Macro F1 on Tiny ImageNet compared to CIFAR-10? Why does Tiny ImageNet present a vastly steeper generalization challenge (considering 200 fine-grained categories, 64x64 spatial complexity, and reduced training samples per class)?

A2: With data augmentation:

Tiny ImageNet Validation Accuracy = 34.24%

CIFAR-10 Validation Accuracy = 81.55%

Tiny ImageNet Macro F1 = 32.37%

CIFAR-10 Macro F1 = 81.53%

Both validation accuracy and macro F1 see substantial decrease when training on Tiny ImageNet.

Tiny ImageNet presents Alexnet with a far more complicated and extensive training challenge that CIFAR-10. First, the Tiny ImageNet's training set is double the size of CIFAR-

10, meaning that many more epochs are required to allow the network to fully generalize. Second, the number of images per class is much lower in Tiny ImageNet than CIFAR-10. Since there are 200 total classes in the dataset, data for each class is scarcer (100000/200 = 500). Third, the feature space is much more complex, and which introduces the possibility of overlap between classes. Classifying classes close together in the feature space thus adds to the difficulty of training. Finally, the images have a 64x64 resolution in Tiny ImageNet, which is double the dimensions of the images in CIFAR-10. As such, there are now 4096 pixels per image instead of the 1024 per image in CIFAR-10.

Q3: Did the Tiny ImageNet validation loss diverge or plateau earlier in the training schedule compared to CIFAR-10? Explain this behavior using your generated loss curves.

A3:

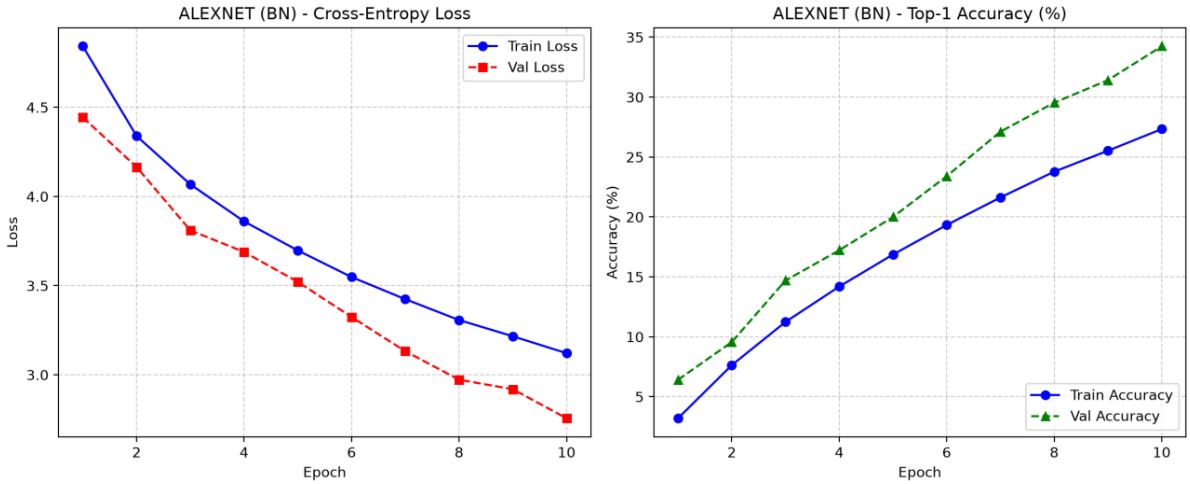


Figure 21: Train and validation curves for Alexnet with BN and data augmentation, trained on Tiny ImageNet dataset

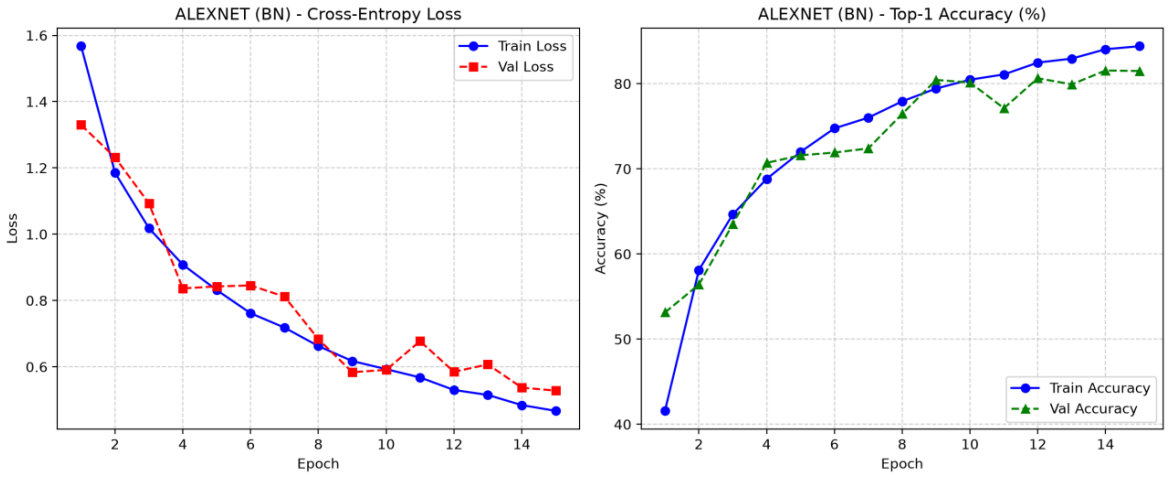


Figure 22: Train and validation curves for Alexnet with BN and data augmentation, trained on CIFAR-10 dataset

The Tiny ImageNet validation loss never plateaued or diverged but rather continued steadily on a downward trend. Since the training set was heavily augmented, this likely brought the training loss above the validation loss as observed in Figure 21. The CIFAR-10 validation curve also continued downward but it seemed to being to plateau towards the end of training at epoch 14.